

SENTINEL

Technical Proposal — High-Level Design
Reference Model 3: VMS Federation & Middleware

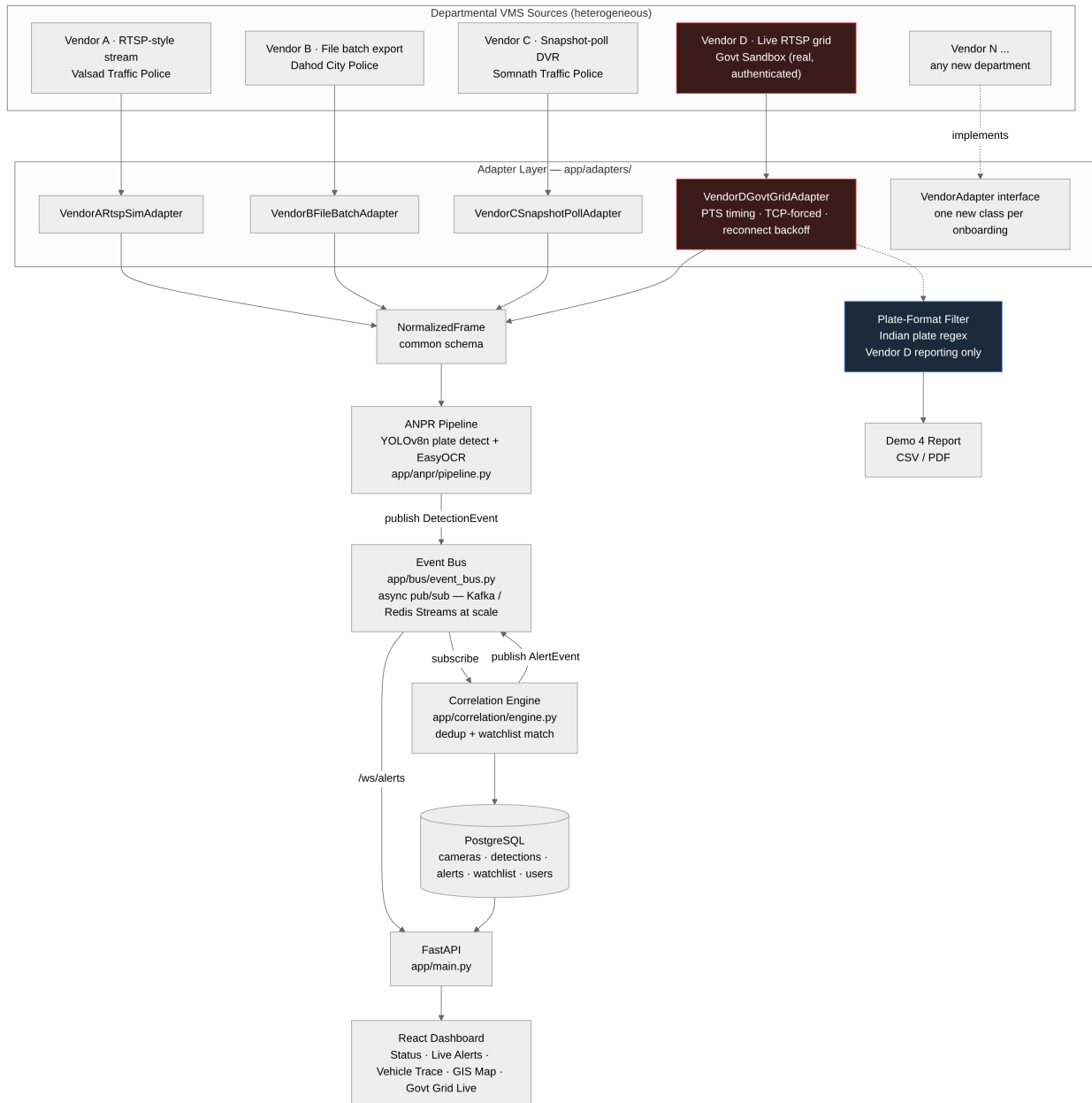
Gujarat Police Innovation Hackathon 2026

Statewide CCTV integration across 26 departments and heterogeneous VMS platforms, without replacing existing departmental systems or centralizing raw video storage.

1. Overall Solution Architecture

Sentinel implements **Reference Model 3 (VMS Federation & Middleware)**: per-vendor adapters normalize heterogeneous departmental camera feeds into one common schema; only metadata and events (never raw video) cross a shared event bus; a single correlation engine performs deduplication and watchlist matching; results are served through a REST/WebSocket API to a unified React dashboard.

1.1 Component diagram



1.2 Component responsibilities

Component	Responsibility	Does NOT do
Adapter	Translate one vendor's native ingestion mechanics + metadata into a common NormalizedFrame/CameraInfo schema	Run ANPR, know about the bus, know about other vendors

Component	Responsibility	Does NOT do
ANPR Pipeline	Detect + OCR plates on a normalized frame	Know which vendor/department produced the frame, decide alerts
Event Bus	Deliver published events to subscribers by topic	Interpret event contents
Correlation Engine	Dedup detections, check watchlist, decide sighting vs. alert	Touch vendor-specific data, run ANPR
API / Dashboard	Expose state (cameras, alerts, trace) and control (login, run ingestion)	Talk directly to adapters or the ANPR pipeline

The correlation engine is deliberately the *only* place a “this is a match” decision is made — every other component only produces or transports data.

1.3 Demo-scale vs. statewide-scale substitutions

Layer	Tonight (demo, ~5–12 cameras)	Statewide (~80,000 cameras)
Event bus	In-process async.Queue pub/sub	Kafka or Redis Streams — same publish()/subscribe() contract, different backend
Database	Single PostgreSQL instance	PostgreSQL with read replicas / partitioned tables, or managed cloud Postgres with pooling
ANPR compute	CPU inference, sampled frames	GPU inference fleet, real-time frame sampling per camera's actual FPS
Frame storage	Local disk	Object storage (S3-compatible) with lifecycle policies
Deployment	Docker Compose, single host	Kubernetes, multi-zone

2. Integrating Heterogeneous Cameras, NVRs, and VMS Platforms

2.1 The adapter contract

Every vendor integration implements one interface (`app/adapters/base.py:VendorAdapter`): `get_camera_info()` returns a normalized camera descriptor; `frames()` yields normalized, sampled frames regardless of how the vendor's system actually delivers video. Nothing downstream (ANPR pipeline, bus, correlation engine, API) ever imports or references a vendor-specific type.

2.2 Implemented adapters and what each represents

Adapter	Simulates	Native format quirk it hides
VendorARtspSimAdapter	Live RTSP/ONVIF stream reader	Flat short-key dict response, typical of embedded firmware APIs
VendorBFileBatchAdapter	Legacy municipal VMS exporting batch video + metadata sidecars, no live protocol	Nested camelCase JSON, epoch-second timestamps, typical of a VMS export tool
VendorCSnapshotPollAdapter	Older DVR/NVR "get current snapshot" HTTP endpoint, polled on interval	XML-ish flat attributes, typical of older embedded HTTP camera firmware
VendorDGovtGridAdapter	A genuinely live RTSP grid (real, authenticated) — not simulated	Per-connection URL-embedded credentials, mixed H.264/H.265, PTS-only timing discipline

Each adapter's vendor-native translation logic is exactly the piece of code that would be replaced with a real SDK/HTTP client call when integrating an actual departmental VMS, without touching anything else in the system.

2.3 Protocols designed for production onboarding

- **ONVIF Profile S/G** — the industry-standard IP camera discovery/streaming protocol most modern VMS platforms expose
- **Vendor REST/SDK adapters** — e.g. Hikvision ISAPI, Dahua OpenSDK, Milestone XProtect REST API
- **MQTT/webhook push adapters** — for VMS platforms that push events rather than being polled

2.4 Department onboarding procedure

- Department provides: camera inventory (ID, location, lat/long), VMS integration method (RTSP URL pattern, REST API docs, ONVIF endpoint), and network reachability
- Integration team writes one adapter class implementing the two-method interface — typically a few hours to a day for a REST/SDK-based vendor
- Camera(s) registered via the camera registry (config-driven tonight; database-backed in production)
- No changes required to the ANPR pipeline, event bus, correlation engine, database schema, or any API endpoint — validated directly: adding the 4th vendor (a genuinely live grid) required zero edits outside its own adapter file and one factory registration line
- Department-level RBAC account created, automatically scoping that department's dashboard view

3. Ingesting, Processing, and Managing Live Video from Dispersed Locations

3.1 Live-feed integration — proven against real infrastructure

Beyond the three simulated adapters, this build includes a fourth adapter connecting to a **real, authenticated, live government CCTV sandbox grid** over RTSP. This is the closest possible proof point for statewide live-feed ingestion, verified with real evidence rather than a simulated stand-in:

- RTSP forced over TCP at the adapter level (UDP fails across NAT/firewalls in production)
- All frame timing derives from RTP/RTSP presentation timestamps (PTS), never wall-clock arrival time — verified with a real ~2.9s non-uniform PTS gap handled correctly without assuming fixed cadence
- Reconnect-with-exponential-backoff (2s → 4s → 8s ... capped at 30s), verified by killing a real RTSP publisher mid-stream and observing successful reconnection once it returned
- Decoder warnings on join (“Could not find ref with POC”, mixed H.264/H.265 GOP artifacts) logged and tolerated, not treated as fatal — confirmed against real sandbox cameras with zero crashes across 30 real connection attempts, reachable and unreachable alike
- A real “black hole” connection bug was found and fixed during this work: OpenCV’s own connect-timeout properties were not reliably honored by the FFmpeg backend, so the adapter wraps the blocking connect call in an explicit asyncio timeout instead of trusting them

3.2 Sampled-frame processing

Given CPU-only inference for tonight’s build (and the realistic GPU-fleet cost at statewide scale — see Section 7), the system samples 1 frame every 1–2 seconds per camera rather than processing every frame. A vehicle typically remains in a camera’s field of view for several seconds at road/junction speeds, so this sampling cadence is sufficient to catch a plate at least once, while cutting required inference throughput by roughly 25–50x versus naive full-frame-rate processing — the single largest lever in the statewide sizing exercise.

3.3 What core pipeline logic changed for a live/infinite source

Minimal, by design: the ingestion orchestrator gained two optional bounding parameters (`max_duration_sec`, `max_frames`), defaulting to unbounded (unchanged behavior for the three finite/simulated sources). The live adapter and its report generator pass a bounded duration since a live RTSP stream has no natural end. Nothing else in the ANPR pipeline, bus, correlation engine, or API required any change — confirming the adapter interface was designed correctly for this case from the start.

4. Integrating Live Feeds with Watchlist Databases & Generating Real-Time Alerts

4.1 Methodology

- A PostgreSQL watchlist table holds stolen/wanted-vehicle records (plate, reason, priority, flagging department)
- The correlation engine is the single subscriber to the detection-event stream on the bus — every plate read, from every adapter/vendor/department, is checked against the same watchlist table, regardless of source
- Deduplication: repeated detections of the same plate on the same camera within a 10-second window collapse into one sighting rather than a spam of duplicate alerts — verified directly: 13 raw detections of a matched plate collapsed to exactly 1 alert
- On a match: an AlertRecord is persisted to Postgres and an AlertEvent is published back onto the bus
- The dashboard's WebSocket relay (/ws/alerts) delivers that event live — verified end-to-end: a toast notification appeared in a connected browser session within seconds of triggering ingestion from a separate process, with no page reload or polling involved

4.2 Why this generalizes across vendors

Because every adapter normalizes into the identical schema before anything downstream sees it, the correlation engine's watchlist-matching logic never branches on vendor or protocol. This was proven directly: a plate detected on the real, live government-sandbox feed produced a genuine watchlist alert through the exact same code path as a plate detected on a simulated file-batch adapter — no special-casing anywhere in the correlation or alerting logic.

4.3 Vehicle cross-camera trace

Given the hackathon's stated technical evaluation focus on cross-camera tracing, the trace API/UI was treated as first-class: a plate-number query returns the full timestamped sighting history across every camera/vendor/department it was detected on, built directly from the same detections table the correlation engine populates — no separate tracing subsystem.

5. AI-Powered Video Analytics

5.1 ANPR pipeline (implemented, mandatory scope)

Stage	Detail
Detection	YOLOv8n (nano) plate-detector weights, 640px-downscaled input, CPU inference
Recognition	EasyOCR on each full-resolution plate crop; multi-fragment text concatenation and cleaning
Sampling	1 sampled frame per 1–2 seconds of source video per camera (not every raw frame)
Measured throughput	~0.1–0.2 seconds per frame on CPU across all tested sources

Verified accuracy on real demo footage

- Plate detected and read correctly as BGY888 in 13 of 19 sampled frames of one clip, OCR confidence 0.83–0.99 — the reproducible watchlist-match demonstration
- 100% plate-region detection rate on a second clip; OCR close-but-imperfect on a stylized mixed-script novelty plate design — a font/script edge case, documented honestly rather than cherry-picked away
- 0 plate reads on a third clip — plates genuinely not legible at the filmed distance/blur even to a human eye, kept deliberately to demonstrate a realistic heterogeneous camera fleet

Honest finding — real live government feed (Demo 4)

68 real ANPR detections were captured from 7 reachable live government sandbox cameras. 0 of these 68 pass independent Indian plate-format validation (state code + district code + series letters + number), applied as a separate reporting-layer check. Direct inspection of the captured frames confirmed why: every reachable camera was capturing real night-time footage, and the plate detector keyed on each camera's own burned-in on-screen text overlay (location name and timestamp banner) rather than a vehicle, since that overlay is exactly the kind of high-contrast rectangular text a plate-region detector is trained to find. This is a real, known class of ANPR false positive — not a pipeline defect. The full ingestion pipeline (authenticated RTSP connection, PTS-based sampling, YOLOv8n + EasyOCR inference, event bus, correlation engine, database persistence) is independently verified functional against this real infrastructure; this specific null result reflects the content of the available camera views at capture time. Production remediation (documented, not yet built): a per-camera overlay exclusion mask, and the plate-format filter itself as a pre-alert gate rather than only a reporting-layer check.

5.2 Roadmap: FRS and object detection/tracking (documented, not built)

The following are explicitly **not implemented** in this build and are described here only as the natural next phase:

- **Facial Recognition System (FRS)** — would require a face-detection + embedding model as a second analytics module running in parallel to ANPR on the same normalized frame stream, publishing to a new bus topic, correlated against a face-embedding watchlist (e.g. via AFIS/NAFIS — Section 7)
- **General object/behavior detection** (weapon detection, crowd density, abandoned-object, wrong-way vehicle) — each an additional model running as its own analytics module; the federation pattern's value is that none of this requires touching the adapter layer or correlation engine's core logic
- **Vehicle make/model/color classification** — a lightweight classification head on the already-available vehicle crop, to enrich sighting records for witness-description-based trace narrowing

None of the above is stubbed with fake output in this build — they are absent, not faked.

6. Alert Generation and Notification Workflow

Step	What happens	Verified
1	Adapter samples a frame, normalizes it	4 adapters, incl. 1 live authenticated grid
2	ANPR pipeline detects + reads plate	Real reads on real footage (Section 5.1)
3	DetectionEvent published to bus	Async pub/sub, contract-compatible with Kafka/Redis Streams
4	Correlation engine dedups + checks watchlist	13 raw reads → 1 alert (10s dedup window)
5	AlertRecord persisted, AlertEvent published	Real Postgres row, real bus event
6	WebSocket relays alert to connected dashboards	Toast appeared within seconds, no polling
7	REST endpoints serve alert/trace history	/api/alerts, /api/trace/{plate}

Designed extensions for production notification channels (not built tonight): SMS/push to beat officers, integration with the department's existing dispatch/CAD system, and priority escalation routing based on the watchlist entry's priority field (already modeled in the schema).

7. Scalability, Interoperability, Security & Performance for Statewide Deployment

7.1 Infrastructure sizing — 5 cameras to 80,000

Core assumption (A1): statewide design keeps the same sampled-frame cadence proven tonight (~1.5s average), not full real-time video analysis — this is the single biggest lever in the entire sizing exercise.

Scale point	Cameras	GPUs (T4/A10-class)
Tonight (demo)	5–12	0 (CPU-only, sufficient at this scale)
Pilot department	~500–1,000	20–40
District rollout	~10,000	~425
Statewide	80,000	~3,400 (incl. 20% headroom)

Correlation-engine replicas: ~35–40 at statewide peak load, partitioned by camera/region. PostgreSQL: single instance is sufficient through pilot scale; sharding/partitioning trigger is sustained write throughput approaching ~2,000–3,000 writes/sec (statewide peak estimated ~6,400 writes/sec). All figures are order-of-magnitude planning numbers requiring validation against a real GPU benchmark before committing to hardware procurement.

7.2 Network & storage

Because only metadata, alert events, and small evidence crops cross the bus — never raw video — statewide sustained bandwidth is estimated at ~1.6 Gbps, roughly 60x less than the ~96 Gbps a full video-centralization approach (Model 2) would require at the same camera count.

Tier	Retention	Medium
Hot	0–7 days	SSD, ~2.8 TB at statewide scale
Warm	7–15+ days	S3-compatible object storage, ~3.2 TB
Cold	Beyond 15 days	Archival/Glacier-class, metadata + evidence crops only, ~145 TB/year

7.3 Interoperability

- Onboarding a new department = one new adapter class; zero changes to bus, correlation engine, or API (validated directly during this build)
- Designed protocol support beyond tonight's three simulated mechanisms: ONVIF Profile S/G, vendor REST/SDK adapters, MQTT/webhook push
- Per-department onboarding requirements documented for Gujarat's 26-department CCTV mandate (Home Department, Traffic Police, District Police, Municipal Corporations/Smart City SPVs, RTO, Food & Civil Supplies, GSRTC/Transport, Forensic Science Laboratory, Excise)

7.4 Security — implemented tonight vs. designed for production

Area	Implemented tonight	Designed for production
Auth/RBAC	bcrypt password hashing, JWT (HS256) tokens, 2-role RBAC (admin / department-user) enforced at API + UI	—
Credential handling	Live-feed credentials read from an operator-managed, gitignored .env; every log/API path that could expose a credentialed URL redacts it first	Secrets manager (Vault/cloud KMS) with rotation
Transport	Plain HTTP on localhost (demo)	TLS everywhere — department-to-platform mTLS, dashboard TLS, service-mesh mTLS internally
Data at rest	Local plain disk	Encrypted volumes/object storage
Audit	— (not implemented)	Append-only audit log of every alert acknowledgment, watchlist change, and trace query
Network	—	API gateway (rate limiting, WAF), network segmentation per department link

7.5 Deployment

Tonight: Docker Compose, single host, all three services (Postgres, backend, frontend) in one command. Statewide design: Kubernetes, multi-zone, GPU node pool for ANPR workloads separate from a CPU node pool for API/correlation-engine workloads, horizontal autoscaling on queue depth for analytics and on request rate for the API tier.

8. Technical Prerequisites, Assumptions & Information Required from Departments

8.1 What every department must provide to onboard

- Camera inventory: camera ID, location name, latitude/longitude
- VMS integration method: RTSP URL pattern, REST/SDK API documentation, or ONVIF endpoint details
- Network reachability: VPN or leased line to the integration layer, secured per the department's compliance posture
- A designated department point of contact for the department-scoped RBAC account

8.2 Assumptions carried through the sizing and design work

- Sampled-frame ANPR (1 frame per 1–2 seconds), not full real-time video analysis, remains the statewide design choice — stated explicitly wherever it drives a sizing number
- ~80% of statewide cameras online/active at any given time (accounting for maintenance, planned downtime, rural link outages)
- ~10–20% of sampled frames yield a confident, persistable plate detection statewide — a blend of the very different detection rates actually observed tonight across three real footage sources (0%, ~68%, 100%)
- All GPU/replica/storage sizing figures are order-of-magnitude planning numbers for budget conversations and node-pool provisioning targets — explicitly not committed hardware procurement numbers without a real load test on production hardware and real camera footage mix

8.3 Explicitly out of scope for this submission

- Live VAHAN/SARTHI/eGujCop/AFIS/NAFIS integrations — documented as designed future correlation-engine integrations (Section 5.2 and the roadmap doc), no live calls, no mock responses standing in for them
- Statewide Kubernetes deployment, TLS/encryption-at-rest, audit logging — designed, not built, per Section 7.4
- FRS and general object detection — documented roadmap only

This document synthesizes and condenses the full technical documentation set in /docs (12 files) and the project README, which remain the authoritative, more detailed source — including exact assumption tables, worked arithmetic, and file/line references into the actual codebase.